

Azure Databricks Notes

1. Apache Spark Cluster Architecture

Apache Spark is an open-source, distributed cluster-computing framework designed for fast processing of large datasets. At the core of Spark is a master-worker architecture comprising a **Driver Node**, a **Cluster Manager**, and multiple **Worker Nodes**.

The **Driver Node** maintains the application state, runs the main program, initializes the SparkSession, translates code into physical execution plans, and coordinates tasks across workers. If the Driver Node crashes, the entire Spark application fails immediately, regardless of worker health.

Worker Nodes host **Executor processes**. Executors run individual computational tasks in parallel, store cached data in RAM/disk, and report status back to the Driver.

During execution:

1. Code transformations construct a logical **Directed Acyclic Graph (DAG)**.
2. The DAG Scheduler divides the logical plan into execution **Stages** based on data shuffling boundaries.
3. The Task Scheduler breaks stages into discrete **Tasks** sent to executor slots on Worker Nodes.

2. Databricks Platform & Multi-Cloud Footprint

Databricks is a managed, unified enterprise platform built around Apache Spark. It abstracts cluster management, environment configuration, and scaling while providing native integrations with the major public cloud providers:

- **Azure Databricks:** Deeply integrated with Microsoft Azure services (Microsoft Entra ID, Azure Data Lake Storage Gen2, Azure Key Vault, Azure Data Factory).
- **Databricks on AWS:** Integrated with AWS Glue, S3, IAM, and QuickSight.
- **Databricks on GCP:** Integrated with Google Cloud Storage, BigQuery, and Google Cloud Identity.
- **Control Plane:** Hosted in Databricks' account. Contains the web interface, workspace management, notebook revision histories, and job scheduling state.
- **Data Plane:** Resides directly inside your company's cloud subscription/virtual network (VNet). Virtual machine nodes and raw storage remain entirely within your subscription boundary, ensuring strict data residency compliance.

3. Azure Data Lake Storage Gen2 (ADLS Gen2) Integration

Azure Data Lake Storage Gen2 (ADLS Gen2) combines standard Azure Blob Storage with a **Hierarchical Namespace (HNS)**. HNS organizes objects into true directory trees rather than simulating folders via key prefixes.

This structure makes directory-level modifications (renaming, deleting, or moving folders containing millions of files) atomic $O(1)$ pointer operations instead of $O(n)$ read-copy-delete batch cycles, drastically decreasing Spark batch job latency.

To connect Databricks to ADLS Gen2, workspaces use storage account endpoints mapped through specific container paths:

```
abfss://<container-name>@<storage-account-name>.dfs.core.windows.net/<path>
```

4. Azure Databricks Workspace & Compute Architecture

A **Databricks Workspace** is the portal where data teams collaborate. Compute clusters inside a workspace are provisioned in two main operational modes:

Cluster Compute Types

1. **All-Purpose Clusters:** Interactive clusters used for exploratory data analysis, notebook debugging, and ad-hoc analytics. Manually created and terminated by users.
2. **Job Clusters (Ephemeral):** Purpose-built clusters launched automatically by Databricks Workflows to execute scheduled pipelines. The cluster provisions when the job triggers and terminates immediately upon task completion, reducing DBU overhead by 30-50%..
3. **Serverless Compute:** Fully managed compute layers where instances start in <10 seconds, removing manual node selection, warm-up delays, and auto-scaling configuration.

Cluster Optimization Modes

- **Single-Node vs. Multi-Node:** Single-node configurations run Driver and Worker processes on one virtual machine—ideal for non-distributed workload execution (like standard pandas scripts) without distributed overhead.
- **Photon Engine:** A high-performance C++ vectorised query engine native to Databricks Runtime. It accelerates Spark DataFrame and SQL operations on large datasets, particularly for workloads with heavy scanning, aggregations, and joins.

5. Magic Commands & Polyglot Notebook Execution

Databricks notebooks support multi-language cell execution within the same notebook using cell-level **magic commands**.

```
# Default Notebook Language: Python
df = spark.read.format("delta").load("/mnt/gold/sales")
df.createOrReplaceTempView("global_sales")

-- Switch context to Spark SQL using %sql
%sql
SELECT region, SUM(amount) AS total_revenue
FROM global_sales
GROUP BY region;

-- Execute shell commands on Driver VM using %sh
%sh
ls -la /dbfs/mnt/

-- Render documentation using %md
%md
# Weekly Executive Sales Report
```

Available language magic commands include %python, %sql, %scala, %r, %sh, and %md.

Cross-Language Variable Boundary: Local variables defined in language-specific runtimes (such as a local Python variable `x = 10`) do not cross runtime environments directly into %sql or %scala contexts. To share data structures between Python and SQL, register DataFrames as Temporary Views (`df.createOrReplaceTempView()`).

6. Databricks File System (DBFS) Architecture & Security

The **Databricks File System (DBFS)** is a virtual file system layer mounted over cloud object storage. It maps local Unix file paths (e.g., `/dbfs/mnt/data`) directly to object storage targets (`abfss://...`).

Virtual File Path: `/dbfs/mnt/landing/orders.csv`

| (Mapped under the hood)



Physical Cloud Path: `abfss://landing@storacc.dfs.core.windows.net/orders.csv`

Critical Security Boundaries

- **Root DBFS (dbfs:/):** The default root storage automatically provisioned with every Databricks workspace. All workspace users have read/write access to root DBFS by default. Storing production files, secrets, or sensitive PII data in root DBFS presents a significant security risk.
- **Mount Point Deprecation:** Mount points created via `dbutils.fs.mount()` expose storage access workspace-wide. Modern enterprise standards replace legacy DBFS mounts with **Unity Catalog Volumes and External Locations**, providing fine-grained access governance.

7. Accessing ADLS Gen2 from Databricks using Service Principals

A **Service Principal** acts as an automated identity (app account) within Microsoft Entra ID. Authenticating Databricks to ADLS Gen2 via a Service Principal prevents individual user credentials from being hardcoded into notebooks.

Implementation Steps

1. Register an Application in **Microsoft Entra ID** to generate an Application (Client) ID and Tenant ID.
2. Create a Client Secret within the registered application.
3. Grant the Service Principal the **Storage Blob Data Contributor** role on the target ADLS Gen2 storage container.
4. Configure the SparkSession inside Databricks using OAuth 2.0 credentials:

```
service_principal_id = dbutils.secrets.get(scope="keyvault-scope", key="sp-client-id")
service_principal_secret = dbutils.secrets.get(scope="keyvault-scope", key="sp-client-secret")
tenant_id = dbutils.secrets.get(scope="keyvault-scope", key="sp-tenant-id")
directory_name = "finance-container"
storage_account_name = "adlsgen2finance"
```

```
spark.conf.set(f"fs.azure.account.auth.type.{storage_account_name}.dfs.core.windows.net",
"OAuth")
spark.conf.set(f"fs.azure.account.oauth.provider.type.{storage_account_name}.dfs.core.windows.net",
"org.apache.hadoop.fs.azurebfss.oauth2.ClientCredsTokenProvider")
spark.conf.set(f"fs.azure.account.oauth2.client.id.{storage_account_name}.dfs.core.windows.net", service_principal_id)
spark.conf.set(f"fs.azure.account.oauth2.client.secret.{storage_account_name}.dfs.core.windows.net", service_principal_secret)
spark.conf.set(f"fs.azure.account.oauth2.client.endpoint.{storage_account_name}.dfs.core.windows.net",
```

```
f"https://login.microsoftonline.com/{tenant_id}/oauth2/token")
```

```
df =  
spark.read.load(f"abfss://{directory_name}@{storage_account_name}.dfs.core.windows.net/raw_data.parquet")
```

8. Databricks Utilities (dbutils), Widgets, and Secrets

Databricks provides the built-in dbutils library to handle environment tasks, filesystem operations, parameter input UI, and secret management.

dbutils.fs (Filesystem Utilities)

Interacts directly with DBFS and cloud storage:

```
dbutils.fs.ls("/mnt/landing/")          # List directory contents  
dbutils.fs.cp("/tmp/source.csv", "/mnt/raw/target.csv") # Copy files  
dbutils.fs.mkdirs("/mnt/gold/aggregated/")      # Create directory  
dbutils.fs.rm("/mnt/bronze/temp_data/", recurse=True) # Remove folder recursively
```

dbutils.widgets (Parameterized Notebooks)

Creates interactive UI input controls or accepts dynamic runtime parameter inputs from orchestrators like Azure Data Factory or Databricks Workflows:

```
# Create input parameters  
dbutils.widgets.text("environment", "dev", "Target Environment")  
dbutils.widgets.dropdown("process_date", "2026-07-30", ["2026-07-29", "2026-07-30",  
"2026-07-31"], "Execution Date")
```

```
# Fetch current parameter values  
env = dbutils.widgets.get("environment")  
proc_date = dbutils.widgets.get("process_date")
```

```
# Remove widgets  
dbutils.widgets.remove("environment")
```

dbutils.secrets (Secrets Management)

Fetches secure credentials directly at runtime without logging plaintext values. When backed by an Azure Key Vault scope, secret updates in Key Vault apply automatically in Databricks

notebooks:

```
db_password = dbutils.secrets.get(scope="akv-production-scope",
key="sql-db-admin-password")
# Note: Any output attempt (e.g., print(db_password)) displays [REDACTED] in the cell display.
```

9. PySpark Data Ingestion & Transformation Mechanics

PySpark processes datasets as distributed collections using **DataFrames**. Spark uses **Lazy Evaluation**—transformations build a logical execution plan, but processing occurs only when an **Action** is called.

Transformations (Lazy - Builds DAG):

```
df.select() —> df.filter() —> df.groupBy() —> df.withColumn()
                                     |
```

Action Triggers Execution:

```
df.count() / df.write.save() / df.collect() <—|
```

Data Ingestion

PySpark supports explicit schema definitions to ensure incoming files adhere to expected datatypes:

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType,
TimestampType
```

```
order_schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("customer_id", StringType(), True),
    StructField("amount", DoubleType(), True),
    StructField("order_timestamp", TimestampType(), True)
])
```

```
df_orders = (spark.read
    .format("csv")
    .option("header", "true")
    .schema(order_schema)
    .load("abfss://raw@adlsgen2.dfs.core.windows.net/orders/*.csv"))
```

Common Transformations

```
from pyspark.sql.functions import col, when, current_timestamp, upper
```

```
df_transformed = (df_orders
    .filter(col("amount") > 10.0)
    .withColumn("customer_id", upper(col("customer_id")))
    .withColumn("order_category",
        when(col("amount") >= 1000, "HIGH_VALUE")
        .when(col("amount") >= 100, "MEDIUM_VALUE")
        .otherwise("LOW_VALUE"))
    .withColumn("processed_at", current_timestamp()))
```

10. Real-Time Scenarios & Streaming Output Modes

Structured Streaming processes real-time data flows using the same DataFrame API as batch processing. Stream queries process append-only or updating state logs continuously.

Output Modes

Structured Streaming defines how transformed stream data is written to target sinks using three distinct output modes:

1. **Append Mode (Default):** Only newly processed rows since the last trigger are written to the sink. Suitable for pipelines where existing rows are never modified (e.g., streaming raw JSON files into Bronze Delta tables).
2. **Complete Mode:** Re-writes the entire result table to the sink every time new data arrives. Required for queries containing stateful aggregations (e.g., continuous COUNT() or SUM() window operations).
3. **Update Mode:** Only rows that were modified or added since the last trigger interval are written to the sink. If the stream contains no aggregations, Update mode behaves identically to Append mode.

Structured Streaming Query Example

```
streaming_df = (spark.readStream
    .format("delta")
    .load("/mnt/bronze/orders_stream"))

stream_query = (streaming_df
    .groupBy("customer_id")
    .count()
    .writeStream
    .format("delta")
    .outputMode("complete")
    .option("checkpointLocation", "/mnt/silver/checkpoints/orders_agg")
    .start("/mnt/silver/customer_order_counts"))
```

11. Delta Lake Architecture & Storage Mechanics

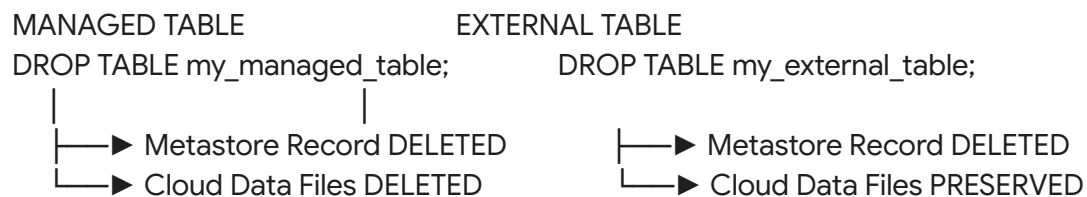
Standard Parquet files lack transaction logs. If a batch write fails halfway through execution, corrupted or partial Parquet files remain in storage. **Delta Lake** resolves this by placing an ACID-compliant transaction log layer on top of standard Parquet files.

Core Capabilities

- **ACID Transactions:** Guarantees that concurrent writes do not leave datasets in inconsistent or partially written states.
- **Schema Enforcement & Evolution:** Prevents bad records from corrupting tables by rejecting writes with incompatible schemas, while allowing controlled schema expansion via `.option("mergeSchema", "true")`.
- **Time Travel:** Retains older versions of underlying physical files, allowing historical states to be queried via explicit version or timestamp pointers.
-

12. External Delta Tables vs. Managed Delta Tables

When creating Delta tables, Unity Catalog or Hive Metastore manages metadata pointers. Table ownership differs depending on table type:



Managed Tables

Created without specifying a custom `LOCATION` path. Databricks manages both table metadata in the catalog and raw Parquet data files within the default metastore storage path.

Executing `DROP TABLE managed_table_name` **permanently deletes both table metadata AND physical cloud storage files.**

```
CREATE TABLE default_catalog.sales_db.managed_orders (  
  order_id STRING,  
  amount DOUBLE  
) USING DELTA;
```


External Tables

Created using an explicit LOCATION parameter pointing to custom cloud storage paths (e.g., ADLS Gen2 path).

Executing DROP TABLE external_table_name **deletes ONLY table metadata records from the catalog**. All physical Parquet files stored in ADLS Gen2 remain intact on disk.

```
CREATE TABLE default_catalog.sales_db.external_orders (  
  order_id STRING,  
  amount DOUBLE  
) USING DELTA  
LOCATION 'abfss://finance@adlsgen2.dfs.core.windows.net/tables/external_orders';
```

13. Delta Log (_delta_log) & CRUD Operations Deep Dive

The _delta_log directory maintains table transaction history through incremental JSON commit files.

Transaction Process

When a mutation operation (INSERT, UPDATE, DELETE, MERGE) executes:

1. Delta Lake reads current active state files referenced in the commit log.
2. Changes write out new Parquet files containing the updated state.
3. A new JSON commit file (e.g., 000002.json) writes to _delta_log/. The commit details which Parquet files were added and which existing Parquet files were marked as removed (tombstoned).
4. Every 10 commits, Delta Lake creates a .checkpoint.parquet file summarizing table state up to that point, preventing readers from having to parse thousands of small JSON files.

_delta_log/000002.json Commit Payload Structure:

```
{  
  "commitInfo": { "timestamp": 1785369600000, "operation": "MERGE", ... },  
  "remove": { "path": "part-00001.parquet", "deletionTimestamp": 1785369600000 },  
  "add": { "path": "part-00002-new.parquet", "size": 154210, ... }  
}
```

Executing MERGE (Upsert) Operations

```
MERGE INTO target_gold_customers AS target  
USING staging_silver_updates AS source
```

```
ON target.customer_id = source.customer_id
WHEN MATCHED AND source.updated_at > target.updated_at THEN
  UPDATE SET
    target.email = source.email,
    target.address = source.address,
    target.updated_at = source.updated_at
WHEN NOT MATCHED THEN
  INSERT (customer_id, email, address, updated_at)
  VALUES (source.customer_id, source.email, source.address, source.updated_at);
```

14. Data Versioning, Time Travel, and Maintenance (VACUUM)

Because Delta mutations write new files while leaving old files in storage, historical table snapshots remain accessible.

Time Travel Queries

```
-- Query snapshot by timestamp
SELECT * FROM gold_orders TIMESTAMP AS OF '2026-07-28 10:00:00';

-- Query snapshot by specific version
SELECT * FROM gold_orders VERSION AS OF 14;
```

Restore tables to prior states using the RESTORE command:

```
RESTORE TABLE gold_orders TO VERSION AS OF 12;
```

Permanent File Cleanup (VACUUM)

Accumulating unreferenced Parquet files from historical commits increases cloud storage costs over time. The VACUUM command permanently removes data files that are no longer referenced by active transactions and are older than a designated retention threshold.

```
-- Default retention run (removes unreferenced files older than 7 days)
VACUUM gold_orders;

-- Dry run: preview files eligible for removal without deleting them
VACUUM gold_orders RETAIN 168 HOURS DRY RUN;
```

Retention Trap: The default retention period is **7 days (168 hours)**. Overriding safety thresholds to 0 hours (SET spark.databricks.delta.vacuum.parallelDelete.enabled = true; VACUUM gold_orders RETAIN 0 HOURS;) while concurrent stream or batch queries are reading the table can cause active operations to fail with FileNotFoundException.

15. Delta Table Optimization & Z-Ordering

Frequent batch writes and streaming pipelines often generate thousands of small Parquet files (1-5MB each). This degrades read performance due to file system metadata scanning overhead.

File Compaction (OPTIMIZE)

OPTIMIZE merges small files within a table into balanced 1GB Parquet files.

```
OPTIMIZE gold_orders;
```

Multi-Dimensional Data Clustering (ZORDER BY)

ZORDER BY reorganizes table data along specified high-cardinality search columns using space-filling curves. This groups related column values into the same physical files.

```
OPTIMIZE gold_orders ZORDER BY (customer_id, order_date);
```

Without Z-Ordering (Scans All Files): With Z-Ordering (Pulls Targeted File Only):

File 1: Cust A, C, F		File 1: Cust A, A, B
File 2: Cust A, B, E	—▶ OPTIMIZE —▶	File 2: Cust C, C, D
File 3: Cust B, D, F	ZORDER BY	File 3: Cust E, F, F
	(customer_id)	
Query "Cust A" scans 3 files		Query "Cust A" reads ONLY File 1

Z-Ordering allows Spark to execute **Data Skipping** using file-level min/max statistics, bypassing irrelevant files during query execution and significantly reducing read IOPS.

16. Auto Loader (cloudFiles) & Incremental Streaming

Auto Loader is an enterprise data ingestion feature that processes new files incrementally as they land in cloud object storage without needing expensive directory rescans.

```
df_autoloader = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", "/mnt/bronze/checkpoints/orders_schema"))
```

```
.option("cloudFiles.rescuedDataColumn", "_rescued_data")  
.load("abfss://landing@adlsgen2.dfs.core.windows.net/incoming_json/"))
```

```
(df_autoloader.writeStream  
.format("delta")  
.option("checkpointLocation", "/mnt/bronze/checkpoints/orders_write")  
.start("/mnt/bronze/orders_delta"))
```

Auto Loader Processing Modes

1. **Directory Listing (Default):** Tracks newly arrived files by scanning input directory paths incrementally using modification timestamps. Efficient for directories containing up to millions of files.
2. **File Notification Mode:** Automatically provisions underlying cloud notification infrastructure (e.g., Azure Event Grid and Queue Storage) to listen for file-creation events directly. This scales to handle input directories containing tens of millions of files without polling.

Schema Drift Handling (_rescued_data)

If incoming raw payload schemas change unexpectedly (e.g., a source application adds new fields or changes data types), Auto Loader routes unparsed or mismatched data fields into a designated `_rescued_data` column rather than failing the pipeline.

17. Databricks Workflows & Job Orchestration

Databricks Workflows is a native orchestration engine that automates data processing pipelines. It manages task dependencies, scheduling, failure alerts, and compute allocation via Directed Acyclic Graphs (DAGs).

Pipeline Design Best Practices

- **Always Use Ephemeral Job Clusters:** Running scheduled production workflows on shared All-Purpose interactive clusters increases cost and risks job resource contention. Assign tasks to ephemeral **Job Clusters** that launch when triggered and terminate immediately upon completion.
- **Repair & Run Failed Tasks:** If a workflow containing 15 sequential tasks fails at Task 12 due to an external network timeout, Databricks allows you to **Repair and Run Failed Tasks**. This re-executes only the failed node and its downstream dependencies, preserving state and skipping redundant processing for Tasks 1 through 11.